2. Claude Tools

Rice Business 2026

Kerry Back
Rice Business

From Advice to Action

A chatbot talks

- Text in, text out
- It can *tell* you how to compute something, scrape something, check something
- But it cannot do it — it only produces words

Tools let it act

- Run code, use a terminal, search and read the web, drive a browser
- Now it *does* the thing and shows you the result
- This session is a tour of the four capabilities that matter most

Last time: the anatomy of a model. Today: the four tools that turn it from a conversation into a coworker.

Running Code

AI Writes and Runs Code

Why this is the big one

- You describe a task in plain English
- Claude writes the code, runs it, reads the output, and fixes its own errors
- You get the chart, the table, the number — not a recipe for making it

Where the code runs

- In the cloud (a chat), in a virtual machine, or on your own computer
- We cover those three homes in detail next session
- The point today: it runs *somewhere*, and you see the result

Nearly everything useful in this course rides on this one capability — the model that can execute what it writes.

Numbers Need Code

Never trust an AI's arithmetic done "in its head." A language model predicts plausible next tokens — it is not a calculator. A number it *writes* can be confidently wrong.

> The fix is simple and non-negotiable: have it **write and run code** for anything quantitative. Code that runs is checkable; a number in a sentence is not. "Compute this in Python and show me the code" should be reflex.

This single habit removes the most common way AI analysis goes wrong for us.

The Command Line

What a Terminal Is

The text way to drive a computer

- The **terminal** (or “command line,” or “shell”) runs typed commands instead of clicks
- Every install, every script, every deployment happens here
- It looks intimidating; it is just a few dozen commands

Why it matters now

- Claude Code works *in* the terminal, and can run terminal commands for you
- You rarely type these yourself — but you should recognize them
- “Install this,” “run my script,” “push to GitHub” all become terminal commands Claude issues

A Few Commands to Recognize

Look around

- `ls` — list files (Windows: `dir`)
- `cd path` — change folder
- `cat file` — print a file

Run things

- `python script.py` — run a Python file
- `python -m uvicorn app:app` — serve an app
- `pip install pandas` — install a library

On a Mac

- Use `python3` instead of `python`
- The rest is the same
- Claude knows the difference and adjusts

You do not need to memorize these. You need to know that when Claude “does something on your computer,” this is the layer it is working at.

The Web

Search and Fetch

Two different actions

- **Search** returns candidate links for a query — like a search engine the model can call
- **Fetch** reads the text of a specific page and hands it back
- Together: find sources, then read them

Why it matters

- The model's training has a cutoff date; the web is how it gets *current*
- "Find this quarter's earnings and summarize the call" is search + fetch + reasoning
- Every claim comes with a link you can check

These are built-in tools that run on Anthropic's servers — roughly $10 per 1,000 searches through the API, and included in the chat apps.

Reading Documents and Data

Clean HTML tables

A page that is really a table? The model reads it directly into a dataframe

PDFs

Software-generated PDFs read as text; scanned ones need OCR — Claude picks the right approach

Files on your machine

Spreadsheets, CSVs, documents in a folder you open — read, joined, analyzed

“Fetch” is not only web pages. It is the general move of pulling outside information *in* so the model can work with real data instead of its memory.

Computer-Use & the Browser

When a Page Won’t Just Read

The problem

- Many sites build their content with JavaScript after the page loads
- A plain fetch sees an empty shell
- Logins, clicks, and multi-step flows need a *real* browser

The answer: drive a browser

- Tools like **Playwright** open an actual Chrome, click, type, and read the rendered page
- Claude can navigate, fill forms, and extract what appears
- In Code mode it can even take screenshots and *look* at the result

This is “computer-use”: the model operating software the way a person would, when there is no clean data feed to tap.

Which Tool for Which Job

The situation	The right tool
Page is a clean HTML table	Read it straight into a dataframe
Files follow a URL pattern	Download them directly
A documented data API (JSON)	Call the API
Content built by JavaScript	Drive a browser (Playwright)
Login, clicks, multi-step flow	Drive a browser
A specific page’s text	Fetch it

You do not choose — Claude does. But knowing the ladder tells you why some requests are instant and others need the heavier browser tool.

The Capability Map

Capability	What it unlocks
Run code	Real computation, charts, files — not guessed numbers
Terminal	Install software, run scripts, deploy
Web search / fetch	Current information, with citable sources
Browser / computer-use	Sites and apps with no clean data feed

Four moves past the chat box. Everything we build in the coming sessions is a combination of these.

The Shift

A chatbot answers questions. A chatbot **with tools** does the work — it computes, installs, searches, reads, and clicks. The rest of this course lives on the tool side of that line.

Breakout

Questions to Discuss

- What is a task in your work that you currently do by hand in a browser — clicking, copying, pasting?
- Where would “always run code for the numbers” have caught a mistake for you?
- What web source do you check repeatedly that an AI could fetch and summarize for you?
- Where is the line, for you, between letting AI *read* the web and letting it *act* on the web?